

VMware **Exit** Strategy

Complete Technical Guide with hyper2kvm + hypersdk

Broadcom's VMware acquisition has made vSphere unsustainable — 2-12x license increases, perpetual licenses eliminated, partners dropped. This guide provides a complete 5-phase exit plan: discovery, pilot, staging, production migration, and decommission. Every technical step covered with the tools, YAML configs, and automation to execute it.

5-Phase Plan | \$810K Savings/100 VMs | Zero Downtime | Apache 2.0

The Broadcom Crisis

Why every VMware customer needs an exit plan — now.

What Broadcom Changed (2024-2026)

Perpetual licenses eliminated — subscription-only. Forced product bundles (pay for features you don't use). 60% of channel partners dropped. Support response times degraded. Customers locked into annual renewals at Broadcom's pricing. No negotiation leverage.

2-12x

License cost increase
reported by customers

\$0

Perpetual licenses
available (eliminated)

60%

Channel partners
dropped from program

\$0

KVM hypervisor cost
(built into Linux kernel)

Risk Assessment: What Happens If You Stay

Risk	Impact	Timeline	Consequence
License cost escalation	Critical	Already happening	2-12x increase at renewal. Forced into VCF bundle (\$200K+ minimum). Budget shock.
Perpetual license end	Critical	Done	No fallback to owned licenses. Annual renewal required or shutdown.
vSphere 7.x end of support	Critical	2025-2026	No security patches. Compliance violations. Audit findings.
Partner ecosystem collapse	High	Ongoing	Your VMware partner may no longer exist. Reduced support options.

Forced bundle upgrades	High	At renewal	Can't buy just vSphere — must buy full VCF at much higher price.
VMware talent drain	High	12-18 months	VMware admins are retraining on KVM/K8s. Harder to find vSphere staff.
Compliance audit failures	Critical	Next audit	Unsupported hypervisor = audit finding. SOX, HIPAA, PCI-DSS impact.

Bottom line: Every quarter you stay on VMware, Broadcom charges more and delivers less. KVM is free, in the Linux kernel, and isn't going anywhere. The migration tool (hyper2kvm) is also free. The only cost is your time — and the longer you wait, the more that costs too.

The 5-Phase Exit Plan

A structured, risk-minimized approach. Each phase has clear gates before proceeding.

1

Phase 1 Discovery & Assessment (Week 1-2)

Connect to vCenter read-only. Discover all VMs with CPU, RAM, disk, OS, power state. Classify by tier (dev/staging/prod). Identify Windows VMs needing VirtIO. Map network dependencies. Run `h2kvmctl doctor` on target hosts. Estimate total migration time. **Output:** VM inventory spreadsheet + migration complexity report.

2

Phase 2 Pilot (Week 3-4)

Migrate 5-10 non-critical VMs (dev/test). Use zkvm TUI for guided migration. Validate: boot test passes, applications work, network reachable, performance acceptable. Document issues and fixes. Build team confidence. **Gate:** All pilot VMs boot and pass health checks.

3

Phase 3 Staging & Batch Testing (Week 5-8)

Migrate staging/QA environments (20-50 VMs). Use YAML automation for batch processing. Test with real workloads. Set up Prometheus/Grafana monitoring on KVM hosts. Create runbooks for common issues. Validate Windows VMs (SQL Server, AD, Exchange). **Gate:** Staging passes full regression testing.

4

Phase 4 Production Migration (Week 9-16)

Migrate production VMs in planned maintenance windows. Batch 10-20 VMs per wave. Use job scheduling (hypersdk) for off-peak hours. Boot verification on every VM. Keep VMware running in parallel for 30 days. Webhook notifications to Slack/Teams. **Gate:** All production VMs running on KVM for 30 days with zero issues.

5

Phase 5 Decommission VMware (Week 17-20)

After 30-day parallel run, decommission vSphere. Cancel VMware/Broadcom licenses. Repurpose or retire ESXi hardware. Remove vCenter, vSAN, NSX. Document lessons learned. Report savings to leadership. **Result:** \$0 VMware cost from this point forward.

20 Weeks: From VMware Lock-In to Freedom

One engineer with hyper2kvm can migrate 200+ VMs in this timeframe.

Larger estates: add engineers linearly (2 engineers = 400 VMs).

Phase 1 Discovery & Assessment

Know what you have before you move it. hyper2kvm + hypersdk automates the entire discovery.

zkvm TUI: vSphere Discovery

- Open zkvm, switch to vSphere tab (Alt+2)
- Auto-populates from GOVC_* environment variables
- Connect to vCenter — read-only, zero changes
- Discovers all VMs: name, CPU, RAM, disk, power state, guest OS
- Streaming discovery — VMs appear as found
- Filter and sort by any column
- Export inventory to CSV/JSON for analysis

Pre-Flight Validation

- `h2kvmctl doctor` checks 9 areas on target hosts:
- Disk space (sufficient for conversion)
- KVM support (cpu virtualization enabled)
- qemu-img and qemu-nbd installed
- libvirt running and accessible
- virtio-win ISO available (for Windows VMs)
- Network bridge configured
- kubectl + KubeVirt (if deploying to K8s)
- govc installed (for vSphere export)
- **Run on every target host before migration day**

VM Classification Matrix

Tier	Examples	Migration Order	Downtime Window	Special Handling	Risk Level
Dev/Test	CI runners, test DBs, sandboxes	Phase 2 (pilot)	Anytime	None — standard pipeline	Low
Staging/QA	Pre-prod mirrors, load test	Phase 3	Business hours OK	Validate app functionality	Low
Internal Infra	Monitoring, CI/CD, GitLab, DNS	Phase 3-4	Off-hours preferred	Migrate replicas first	Medium
Web/App Servers	Nginx, Apache, Node.js, Java	Phase 4 (early)	Maintenance window	Load balancer failover	Medium
Database Servers	PostgreSQL, MySQL, SQL Server	Phase 4 (middle)	Maintenance window	Stop DB, snapshot, migrate, verify replication	High

Windows AD/Exchange	Domain controllers, Exchange	Phase 4 (late)	Planned window	Secondary DCs first, VirtIO injection, boot test	High
Cluster/HA	Pacemaker, WSFC, RAC	Phase 4 (late)	Planned window	Migrate secondary nodes first, test failover	High

Phase 2-3 Pilot & Staging

Prove it works. Build confidence. Establish runbooks.

Pilot Migration: Step-by-Step with hyper2kvm

#	Step	Tool	Command / Action	Expected Result
1	Pre-flight	h2kvmctl	h2kvmctl doctor	All 9 checks pass (green)
2	Open TUI	zkvm	./zkvm or zkvm	TUI launches, Migration tab active
3	vSphere connect	zkvm	Alt+2, connect to vCenter	VM list appears with metadata
4	Select VMs	zkvm	Space to select 5 dev VMs	Selected VMs highlighted
5	Configure	zkvm	Ctrl+R syncs to Migration tab, review settings	Form auto-populated, deploy targets set
6	Run	zkvm	Ctrl+R to execute	Progress bar in right panel, logs in tab 3
7	Monitor	zkvm	Alt+3 for logs, watch progress	Export → convert → fix → deploy → verify
8	Verify	zkvm	Alt+4 for Libvirt tab, check VMs	All VMs listed, state: running
9	Test apps	Manual	SSH/RDP into migrated VMs, test services	Applications working correctly
10	Document	Manual	Record issues, timings, fixes applied	Pilot report for Phase 3 planning

Or Use CLI Automation (for batch/staging)

```
# batch-staging.yaml – Migrate staging environment
cmd: vsphere
vcenter: vcenter.corp.com
vs_action: export_vm
vs_vm: staging-web-01      # Run for each VM (or use hypersdk batch queue)
output_dir: ./converted
out_format: qcow2
flatten: true
compress: true
regen_initramfs: true     # Linux: rebuild with VirtIO
emit_domain_xml: true     # Generate libvirt XML
vm_name: staging-web-01
memory: 4096
vcpus: 2
```

```
machine: q35
boot_test: true           # Verify VM boots
verbose: true

# Run: sudo h2kvmctl --config batch-staging.yaml
# Repeat for each VM, or use hypersdk job scheduler for batch queue
```

Windows VMs in Staging

Windows Migration Config

```
# win-sql-staging.yaml
cmd: vsphere
vcenter: vcenter.corp.com
vs_action: export_vm
vs_vm: staging-sql-01
output_dir: ./converted
out_format: qcow2
windows: true
guest_os: windows
virtio_win_iso: /usr/share/virtio-win/virtio-win.iso
win_stage: bootstrap
emit_domain_xml: true
vm_name: staging-sql-01
memory: 8192
vcpus: 4
machine: q35
uefi: true
boot_test: true
```

What hyper2kvm Does Automatically

- Exports VMDK from vCenter via govc
- Converts VMDK → QCOW2 with compression
- **Linux VMs:** Rebuild initramfs (VirtIO), fix fstab (UUID), fix GRUB, remove VMware Tools, SELinux relabel
- **Windows VMs:** Inject VirtIO drivers (viosstor, NetKVM, Balloon), fix registry, fix BCD, remove VMware Tools
- Generates optimized libvirt domain XML
- Runs virsh define to register VM
- Starts VM and verifies boot (no BSOD/panic)
- Reports pass/fail per VM

Phase 4 Production Migration

The main event. Planned maintenance windows. Batch processing. Zero data loss.

Production Migration Waves

Wave	Week	VMs	Workload Types	Window	Rollback Plan
Wave 1	Week 9-10	10-20 VMs	Stateless web servers, proxies	Saturday 2AM-8AM	Revert to VMware (still running)
Wave 2	Week 11-12	20-30 VMs	App servers, internal tools	Saturday 2AM-8AM	Revert to VMware snapshot
Wave 3	Week 13-14	20-30 VMs	Database servers (secondary first)	Sunday 12AM-8AM	Promote replica, revert primary
Wave 4	Week 15-16	Remaining	AD DCs, Exchange, critical infra	Sunday 12AM-8AM	Secondary DCs on VMware until verified

Per-VM Production Migration Checklist

#	Step	Action	Verification
1	Pre-migration snapshot	Take VMware snapshot (rollback point)	Snapshot visible in vCenter
2	Application drain	Remove from load balancer, drain connections	No active sessions/connections
3	Database quiesce	Stop DB or put in maintenance mode	No writes in progress
4	Export from vSphere	hyper2kvm exports via govc (streaming progress)	VMDK downloaded, checksum verified
5	Convert + fix	VMDK → QCOW2, initramfs/VirtIO/fstab/GRUB	Conversion completes without errors
6	Deploy to KVM	Generate XML, virsh define	VM registered in libvirt
7	Boot verification	Automated boot test — VM reaches login	No BSOD, no kernel panic, services up
8	Application test	Health check endpoints, smoke tests	HTTP 200, DB queries succeed
9	DNS / LB cutover	Update DNS records, add to load balancer	Traffic flowing to KVM VM
10	Monitor 24h	Watch metrics, logs, alerts	No anomalies in first 24 hours
11	Keep VMware 30 days	Don't delete VMware VM yet (rollback)	VMware snapshot preserved as safety net

Rollback is always safe: hyper2kvm creates a *copy* — it never modifies the source VMware VM. If anything goes wrong, the original VM is untouched. Revert DNS/LB and start the VMware VM. Total rollback time: 5 minutes.

Target Architecture

Four landing zones — choose based on your team's skills and workload requirements.

Option A: Pure KVM + libvirt

- **Best for:** Traditional IT, existing Linux teams
- RHEL/Fedora/Ubuntu + KVM (in kernel)
- libvirt for VM lifecycle management
- Cockpit web UI for browser-based admin
- virt-manager for desktop admin
- Live migration between hosts (shared storage)
- **Cost:** \$0 software
- **Complexity:** Lowest
- **hyper2kvm:** `emit_domain_xml: true`

Option B: K3s + KubeVirt

- **Best for:** Cloud-native teams, edge sites
- K3s: single binary, 512MB RAM
- KubeVirt: VMs as K8s pods
- GitOps with ArgoCD/Flux
- Same tooling for VMs + containers
- Edge-ready (single-node deployable)
- **Cost:** \$0 software
- **Complexity:** Medium
- **hyper2kvm:** `deploy_k8s: true`

Option C: OpenShift Virtualization

- **Best for:** Red Hat shops wanting vendor support
- OpenShift + OpenShift Virtualization
- Red Hat supported KubeVirt
- MTV available (but hyper2kvm is more flexible)
- Enterprise support contracts + SLAs
- Certified, hardened, tested
- **Cost:** \$50K+/yr (OpenShift subscription)
- **Complexity:** High

Option D: Hybrid (Recommended)

- **Best for:** Large estates, mixed requirements
- Critical/simple VMs → KVM/libvirt
- Cloud-native workloads → K3s + KubeVirt
- Edge sites → K3s (single-node)
- hyper2kvm dual-target: same conversion, both outputs
- `emit_domain_xml: true` + `deploy_k8s: true`
- **Cost:** \$0 software
- **Complexity:** Medium
- **Maximum flexibility**

Per-Workload Target Recommendation

Workload Type	Recommended Target	Why	hyper2kvm Config
---------------	--------------------	-----	------------------

Database (PostgreSQL, MySQL, SQL)	KVM/libvirt	Simplest, best I/O control, CPU pinning for SQL licensing	<code>emit_domain_xml: true</code>
Web servers (Nginx, Apache)	KVM or KubeVirt	Stateless — either target works, KubeVirt enables scaling	Either or both
Active Directory DCs	KVM/libvirt	AD prefers stable infrastructure, no K8s complexity	<code>emit_domain_xml: true</code>
Docker/container hosts	K3s + KubeVirt	Already container-native, natural fit for K8s	<code>deploy_k8s: true</code>
Edge site VMs	K3s + KubeVirt	Single-node K3s, GitOps fleet management	<code>deploy_k8s: true</code>
Windows workstations (VDI)	KVM/libvirt	GPU passthrough, SPICE/VNC, simple management	<code>emit_domain_xml: true, uefi: true</code>

Cost Savings

Real numbers: what you save by exiting VMware.

100-VM Estate — 3-Year TCO Comparison

Cost Item	VMware vSphere (Stay)	KVM + hyper2kvm (Exit)	Savings
Hypervisor licensing	\$450,000 (3 yr, post-Broadcom)	\$0	\$450,000
vCenter / management	\$120,000 (3 yr)	\$0 (Cockpit/libvirt)	\$120,000
vSAN licensing	\$180,000 (3 yr)	\$0 (Ceph/local storage)	\$180,000
NSX licensing	\$90,000 (3 yr)	\$0 (Open vSwitch/Linux bridge)	\$90,000
Support contracts	\$75,000 (3 yr)	\$30,000 (optional RHEL support)	\$45,000
Hardware (servers)	\$200,000	\$200,000 (same hardware)	\$0
Migration cost (one-time)	N/A	\$25,000 (labor, 1 engineer, 20 weeks)	-\$25,000
Training	\$0 (existing skills)	\$10,000 (KVM/libvirt training)	-\$10,000
Operations staff	\$450,000	\$400,000 (KVM is simpler)	\$50,000
3-Year Total	\$1,565,000	\$665,000	\$900,000

\$900K

Total savings over 3 years
for 100-VM estate

\$300K/yr

Annual recurring savings
after migration complete

3 months

Payback period
on migration investment

\$0

hyper2kvm license cost
Apache 2.0 open source

SQL Server Additional Savings

If your VMware estate includes SQL Server, KVM's CPU pinning can reduce per-core licensing costs by **20-40%**. For 10 SQL Enterprise servers, that's an additional **\$600K+ saved** over 3 years. Combined VMware exit + SQL right-sizing savings can exceed **\$1.5M**.

Risk Mitigation

How hyper2kvm eliminates every common migration failure.

Risk	Without hyper2kvm	With hyper2kvm	How It Works
Linux VM won't boot	Common — missing VirtIO in initramfs	Auto-fixed	Rebuilds initramfs with virtio_blk/scsi/net in chroot
Windows BSOD	Missing VirtIO storage driver	Auto-fixed	Injects viostor + NetKVM from virtio-win ISO
Wrong root device	fstab uses /dev/sda, KVM has /dev/vda	Auto-fixed	Converts all fstab entries to UUID-based mounts
GRUB boot failure	root= parameter wrong	Auto-fixed	Updates GRUB config, regenerates grub.cfg
SELinux denials	Services fail with permission denied	Auto-fixed	Creates /.autorelabel for full restorecon on boot
VMware Tools errors	Services crash looking for hypervisor	Auto-removed	Removes open-vm-tools packages and services
Network config breaks	Old NIC names persist	Auto-cleaned	Removes vmxnet3/Hyper-V NIC configs, udev rules
Snapshot chain corruption	Delta files depend on parent	Auto-flattened	Flattens to single self-contained QCOW2
Corrupt disk undetected	Silent corruption, discovered late	Pre-validated	VMDK descriptor parsing, extent verification
Scale errors (VM #47 of 200)	Human error under fatigue	Automated	YAML config, batch queue, per-VM boot test

Workload-Specific Migration Strategies

Database Servers

- Stop DB before export (consistency)
- Or take VMware snapshot (crash-consistent)
- Migrate secondary/replica first
- Verify replication on KVM
- Promote KVM replica, then migrate primary

Active Directory DCs

- Never migrate all DCs at once
- Migrate secondary DCs first
- Verify AD replication (`repadmin`)
- Then migrate primary (FSMO holder)
- Test: DNS, Kerberos, Group Policy

Clustered / HA Workloads

- Migrate passive/secondary node first
- Test failover to KVM node
- Then migrate active/primary node
- Pacemaker: update resource config
- WSFC: adjust quorum settings

- VirtIO SCSI for best I/O performance

- VirtIO injection handles boot

- Oracle RAC: use rolling migration

Your VMware Exit Starts Today

5 phases. 20 weeks. \$900K saved per 100 VMs.

Every guest fix automated. Every VM boot-verified. Every rollback safe.

hyper2kvm + hypersdk: the complete VMware exit toolkit. Free forever.

hyper2kvm + hypersdk — Enterprise VM Migration Platform | Apache 2.0 Open Source | github.com/ssahani/hyper2kvm